REPORTE TAREA 1
ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Vicente I. Rodríguez Rogers

9 de abril de 2026

22:03

1. Entorno experimental

Para la realización de la tarea, se utilizó un computador con sistema operativo basado en Linux y Ubuntu 22.04.5 LTS como su distribución. Esta cuenta con un procesador AMD Ryzen 5300U con AMD Radeon Graphics x8, 8 GB de RAM y disco sólido de 256 GB.

Cabe destacar también que los datasets manejados en la tarea, en el caso de sorting, son generados por el algoritmo "sorting/scripts/array generation.py *z* su margen está limitado a 10 números distintos, donde como máximo puede ser de 7 cifras cada uno de ellos y un mínimo de 1 cifra. En el caso de la multiplicación de matrices, también su dataset se obtiene mediante el código "matrix multiplication/scripts/matrix generator.py", cuya capacidad de crear matrices cuadradas es de un máximo de 2^{10} de tamaño.

2. Resultados y Análisis

Mediante la ejecución de los algoritmos en diferentes datasets, se logró observar experimentalmente el funcionamiento de estos y las cualidades que poseen en la práctica. Resulta especialmente interesante destacar aquellos comportamientos que divergen de lo esperado teóricamente. Las pruebas se dividieron en dos grandes testeos: multiplicación de matrices y ordenamiento de datos unidimensionales, cuyos resultados se comentan a continuación.

- **Multiplicación de matrices:** Al ejecutar el programa, se obtuvieron los datos retratados en la Tabla 1. Información que nos permite ya ver algunos comportamientos inesperados, que son mucho más

apreciables si es que se calcula el tiempo promedio ($\bar{T}$) y la memoria promedio ($\bar{M}$) de todos los datos, haciendo uso de las siguientes fórmulas:

$$\bar{T} = \frac{1}{k}\sum_{i=1}^{k} t_i$$

$$\bar{M} = \frac{1}{k}\sum_{i=1}^{k} m_i$$

Con estas expresiones, se consolidó la información presentada en la Tabla 2:

Cuadro 1: Comparativa de resultados: Datos brutos vs. Promedios

Algoritmo	n	**Tiempo** (s)	**Mem** (KB)
naive	1024	1.87985	15872
strassen	1024	51.50330	45440
naive	256	0.01133	4352
strassen	256	1.02331	6272
naive	64	0.00034	3584
strassen	64	0.02053	3712
naive	16	7.54e-06	3456
strassen	16	0.00045	3456

(a) Muestra de resultados brutos significativos

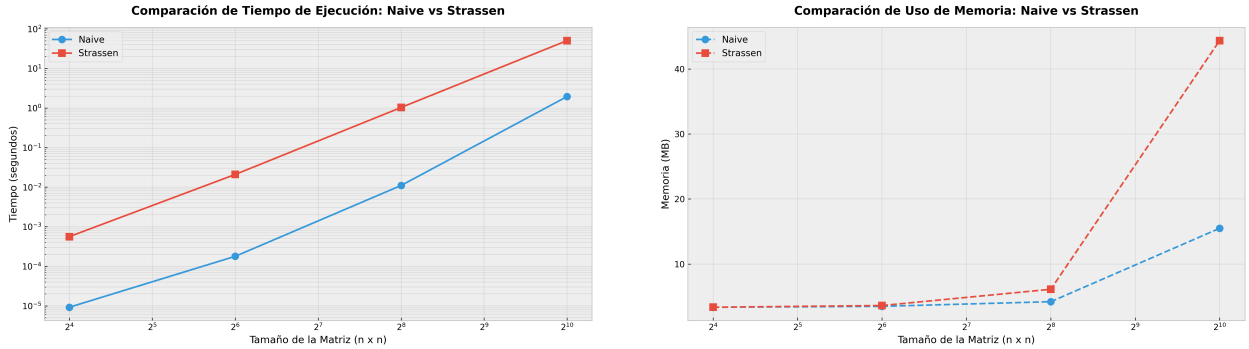
Algoritmo	n	$\bar{T}$ (s)	$\bar{M}$ (KB)
naive	16	0.000009	3427.56
naive	64	0.000178	3569.78
naive	256	0.011011	4302.22
naive	1024	1.945440	15857.78
strassen	16	0.000560	3434.67
strassen	64	0.020883	3697.78
strassen	256	1.030509	6257.78
strassen	1024	50.447661	45418.67

(b) Tiempos y memoria promedio

(En la carpeta `code/matrix_multiplication/data/measurements/results.csv` **se encuentran los datos extendidos de la tabla 1)**

Como se puede apreciar en las tablas, el tiempo promedio del algoritmo de Strassen es significativamente superior (a veces un poco más de un par de cifras significativas) al algoritmo Naive, lo que sorprende ya que en su tiempo teórico este último es de $O(n^3)$, que es peor que el de Strassen con su $O(n^{2,807})$. Y no solo en tiempo Strassen está siendo peor, también en uso de memoria, llegando a ser casi 3 veces más que su competidora, lo que podría señalar cuál es la razón para que su comportamiento temporal se haya disparado.

A continuación se dejan unos gráficos comparativos que permiten visualizar de mejor manera el comportamiento de los algoritmos durante los test.



(a) Comparación de tiempo de ejecución.

(b) Comparación de uso máximo de memoria (KB).

Figura 1: Análisis experimental de complejidad para multiplicación de matrices: Naive vs. Strassen.

De estos graficos destaca el abrupto salto de memoria que hace Strassen, esta usando **3 veces mas memoria** que el algoritmo de Naive y intuitivamente podemos asociar este aumento de consumo a la necesidad de crear constantemente vectores temporales (p1 a p7) lo que a su vez genera un costo administrativo muy alto que problamente sea lo que este impactando en en el tiempo general del algoritmo.

- **Ordenamiento de Arreglos Unidimensionales:** Para esta etapa se evaluaron los algoritmos MergeSort, QuickSort y el estándar de la librería (`std::sort`). Las pruebas abarcaron desde 10^1 hasta 10^7 elementos, utilizando dominios con pocos duplicados ($D7$) y alta frecuencia de estos ($D1$). Al igual que en las matrices, se aplicaron las fórmulas de promedio para sintetizar la información de desempeño:

$$\bar{T} = \frac{1}{k}\sum_{i=1}^{k} t_i \quad ; \quad \bar{M} = \frac{1}{k}\sum_{i=1}^{k} m_i$$

Cuadro 2: Resultados de Ordenamiento: Comparativa de rendimiento y promedios

Algoritmo	N	**Tiempo** (s)	**Mem** (KB)
stdsort	10^7	0.24832	81624
mergesort	10^7	0.83986	81492
quicksort	10^5	4.04266	8404
stdsort	10^5	0.00209	4312
quicksort	10^3	0.00049	3456
mergesort	10	4.74e-06	3328

(a) Muestra de resultados significativos

Algoritmo	N	$\bar{T}$ (s)	$\bar{M}$ (KB)
stdsort	10^7	0.27661	81597
mergesort	10^7	0.79107	81413
quicksort	10^5	2.26443	6145
stdsort	10^5	0.00202	4310
mergesort	10^5	0.00624	4177
stdsort	10^3	2.22e-05	3328
mergesort	10^3	6.80e-05	3328
quicksort	10^3	2.98e-04	3370
stdsort	10^1	5.12e-06	3328
mergesort	10^1	6.45e-06	3328
quicksort	10^1	5.09e-06	3328

(b) Tiempos y memoria promedio (10^1 a 10^7)

(En la carpeta `code/sorting/data/measurements/results.csv` **se encuentran los datos detallados)**

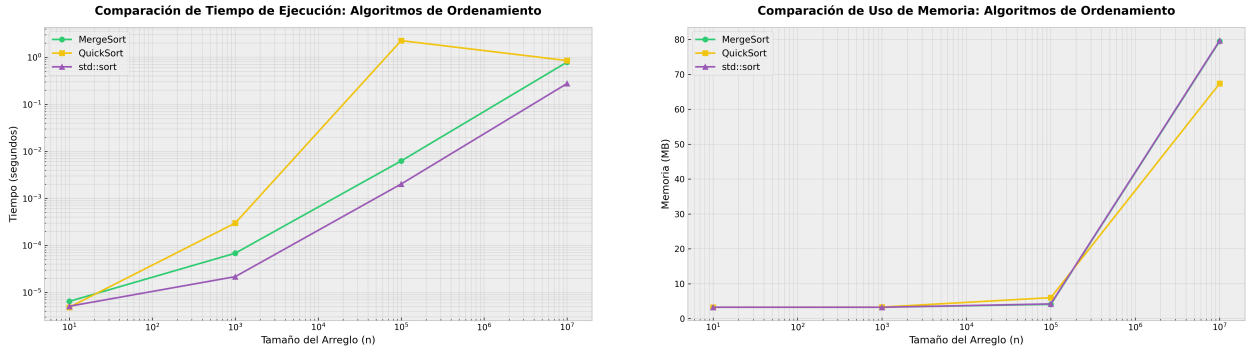
Análisis de Fallo Crítico en QuickSort: Un punto fundamental de este test fue la inestabilidad de la implementación manual de QuickSort. Al ejecutar el `make run`, se obtuvo el siguiente comportamiento en la terminal:

```
Running Benchmarks for 10000000_aleatorio_D1_a...
Segmentation fault (core dumped)
Running Benchmarks for 10000000_ascendente_D1_a...
Segmentation fault (core dumped)
Running Benchmarks for 10000000_descendente_D7_a...
Segmentation fault (core dumped)
```

Listing 1: Log de errores durante la ejecucion de Benchmarks

Este error de **Segmentation fault** es consecuencia directa de un **Stack Overflow**. Debido a que el pivote se selecciona como el último elemento (esquema de Lomuto), en arreglos ya ordenados o con dominios pequeños ($D1$), las particiones resultan extremadamente desbalanceadas. Esto provoca que la profundidad de la recursión alcance los 10^7 niveles, superando el límite de la pila del sistema operativo (8MB), algo que no sucede en **MergeSort** debido a su división siempre simétrica ($O(\log n)$ en la pila).

A continuación se presentan los gráficos comparativos de los algoritmos que lograron completar los sets de datos:



(a) Tiempo de ejecución

(b) Consumo de memoria $O(n)$.

Figura 2: Análisis experimental de ordenamiento: MergeSort vs. QuickSort vs. Introsort.

Finalmente, destaca el desempeño de **std::sort**, el cual implementa **Introsort**. Este algoritmo es superior porque combina la rapidez de QuickSort con la seguridad de HeapSort (al que cambia si la recursión es muy profunda) y la eficiencia de Insertion Sort para tramos pequeños. Esto explica por qué `stdsort` es casi 4 veces más rápido que MergeSort y no falla ante los casos que colapsaron a nuestra implementación de QuickSort.

3. Conclusiones

Luego de aplicar todos los algoritmos, se concluye que, en los algoritmos de ordenamiento, Introsort **(std::sort)** es por lejos el mejor algoritmo. Este obtiene las mejores cualidades de Quicksort en el manejo de memoria y la estabilidad de Merge sort, al no colapsar en situaciones borde. Por otro lado, quedó claro en los algoritmos de multiplicación de matrices que los entornos y condiciones pueden superar la complejidad esperada, y que los factores multiplicativos —que suelen ser ignorados por esta— son también cruciales a la hora de aplicar algoritmos en la práctica. Todo esto se debe al excelente desempeño mostrado por Naive y al incumplimiento de las expectativas que se tenían de Strassen, donde su **"trabajo administrativo"** resultó ser una peor cura que el problema de realizar las multiplicaciones adicionales.